

**SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL –
SENAI
TECNICO EM DESENVOLVIMENTO DE SISTEMAS**

**PEDRO HENRIQUE MOURA DA SILVA
MAYCON SIQUEIRA DE MORAES**

DOCUMENTAÇÃO DE PLANEJAMENTO

**NOVA LIMA – MG
2026**

INTRODUÇÃO:

O presente documento detalha o planejamento das atividades de desenvolvimento e melhorias do trabalho de conclusão do curso técnico em Desenvolvimento de Sistemas, intitulado Vector – Zero E Mission. O projeto foi desenvolvido pelos alunos Pedro Henrique Moura da Silva e Maycon Siqueira de Moraes, sob a orientação do professor Frederico Martins Aguiar.

O planejamento aborda a execução das principais etapas do desenvolvimento, incluindo a criação da estrutura de diretórios, instalação de pacotes, adequação da arquitetura para o padrão MVVM, controle de erros, ajustes no frontend e criação das ViewModels.

CRONOGRAMA DE ATIVIDADES:

TABELA 1 - CRONOGRAMA

Atividade	Responsável	Tempo	Justificativa
Criação dos repositórios e pastas do projeto	Pedro	15	Tendo em vista o tamanho do projeto, acredito ter superestimado o tempo de execução para criar os diretórios, mas acredito que a média gerada pela diferença dos valores é um valor aceitável para conclusão da tarefa.
Instalações de pacotes NuGet	Pedro	8	Devido à necessidade de download dos arquivos das bibliotecas e da quantidade de bibliotecas necessárias para rodar o projeto, acredito que o tempo de 8 minutos seja satisfatório para realizar esta ação.
Ajustes no backend	Maycon	90	Entender a lógica programada e converter do padrão anterior para MVVM demanda uma abstração maior, por esse motivo seria necessário mais tempo para a conversão de um padrão para o outro.

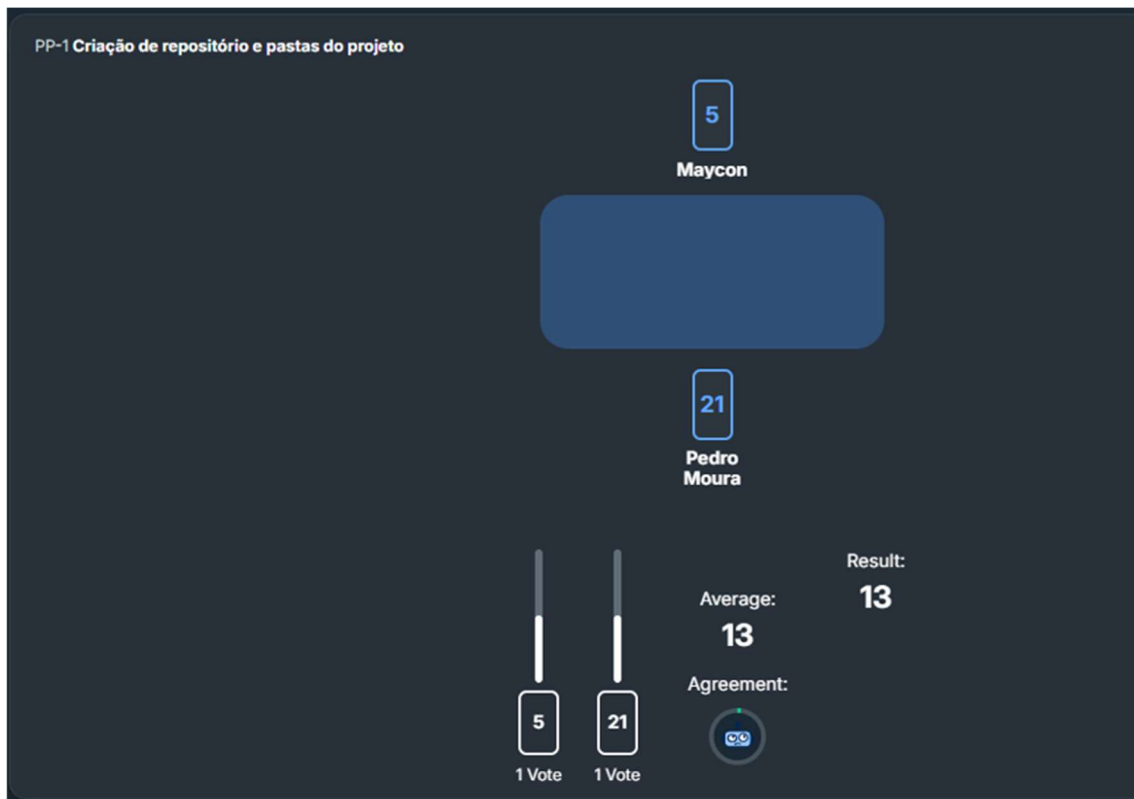
Controle de erros e indentação	Pedro	60	Tendo em vista que a programação do controle de erros envolve testes e codificação, acredito que demanda um tempo maior para validação de todas as possibilidades.
Ajustes no frontend	Maycon	75	O ajuste no frontend depende do alinhamento do binding, da verificação de cada botão de ação e da transferência do mecanismo de ação comum para o método MVVM, por isso demanda um tempo maior para execução.
Criação das view models	Maycon e Pedro	120	A criação das ViewModels envolve a implementação da lógica de apresentação, comandos, notificações de propriedade e integração com os serviços, sendo uma das etapas mais críticas do projeto.

Fonte: Documentação técnica TCC vector 2026

DETALHAMENTO DAS ATIVIDADES:

Criação de repositório e pastas do projeto:

Figura 1 – Votação PlaningPoker



Tempo médio Estimado: 13 minutos

Descrição: Esta atividade consistiu na criação do repositório Git para versionamento do código-fonte, bem como na definição da estrutura de diretórios do projeto. Foram criadas as seguintes pastas principais:

- /Models – Classes de modelo de dados
- /ViewModels – ViewModels do padrão MVVM
- /Views – Interfaces gráficas (XAML)
- /Services – Serviços de negócio e infraestrutura
- /Data – Contexto do banco de dados e migrações
- /Commands – Implementações de ICommand
- /Helpers – Classes auxiliares e extensões
- /Music – Arquivos de áudio para feedback sonoro

Justificativa: A estrutura de pastas bem definida desde o início facilita a organização do código, a localização de arquivos e a manutenção futura. Embora o tempo estimado

tenha sido superestimado, a organização adequada é fundamental para o sucesso do projeto.

INSTALAÇÃO E PACOTES NUGET:

Figura 2 – Votação PlaningPoker



Responsável: Pedro Henrique Moura da Silva

Tempo Estimado: 8 minutos

Descrição: Foram instalados os seguintes pacotes NuGet necessários para o funcionamento do sistema:

TABELA 2 - BIBLIOTECAS

Pacote	Versão	Finalidade
MahApps.Metro	2.4.9	Interface moderna com controles estilo Windows 10/11
MahApps.Metro.IconPacks	4.11.0	Ícones para a interface
LiveCharts	0.9.7	Gráficos dinâmicos e interativos
LiveCharts.Wpf	0.9.7	Interação do LiveCharts com WPF

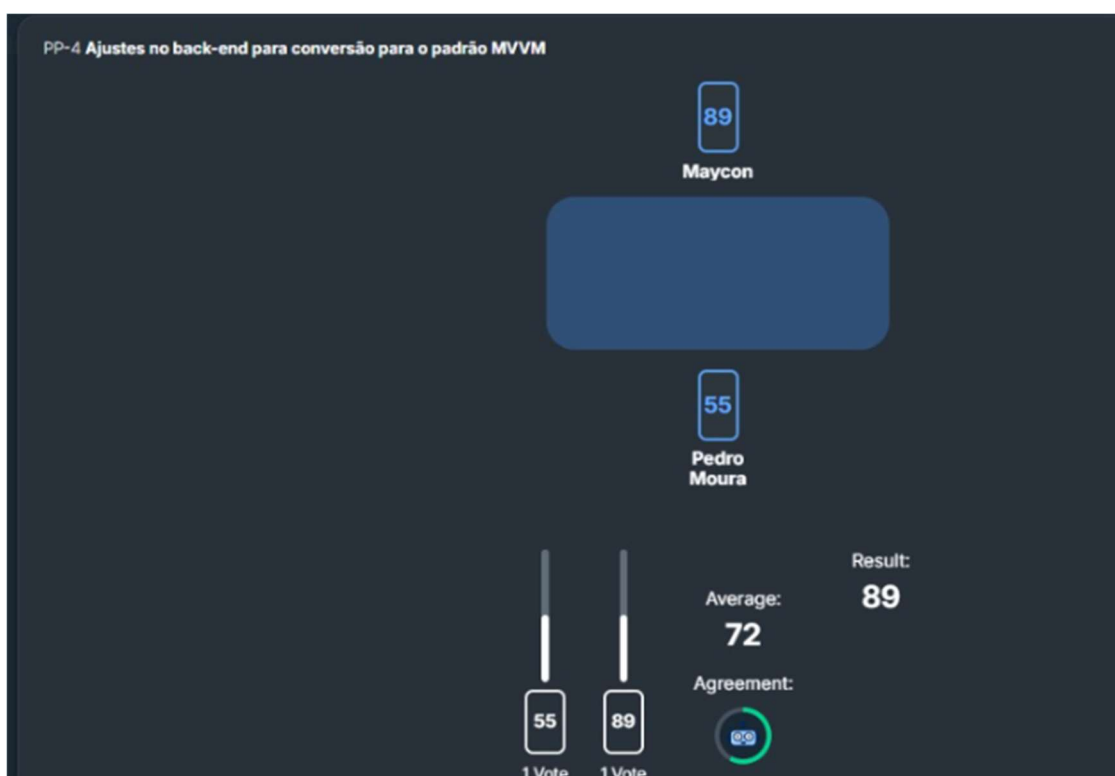
OpenCvSharp4	4.8.0	Processamento de imagens (visão computacional)
OpenCvSharp4.runtime.win	4.8.0	Runtime do OpenCV para Windows
Microsoft.EntityFrameworkCore	6.0.0	ORM para acesso a dados
Microsoft entityFramework.Sqlite	6.0.0	Provedor SQLite para Entity Framework
CsvHelper	30.0.1	Exportação de dados para CSV
Microsoft.Extensions.DependencyInjection	6.0.0	Injeção de dependência

Fonte: Documentação técnica TCC vector 2026

Justificativa: Os pacotes foram selecionados por serem tecnologias maduras, com ampla documentação e suporte da comunidade, garantindo estabilidade e reduzindo o tempo de desenvolvimento.

AJUSTES NO BACKEND PARA O PADRÃO MVVM:

Figura 3 – Votação PlaningPoker



Responsável: Maycon Siqueira de Moraes

Tempo Estimado: 72 minutos

Descrição: Esta atividade consistiu na refatoração do código legado (anteriormente escrito em padrão Code-Behind) para o padrão arquitetural MVVM (Model-View-ViewModel). Foram realizadas as seguintes ações:

Separação de Responsabilidades:

A lógica de apresentação foi movida para as ViewModels.

As Views (XAML) passaram a conter apenas elementos visuais e bindings.

Os Models foram criados para representar os dados do domínio.

Implementação de INotifyPropertyChanged:

Todas as ViewModels herdam de BaseViewModel, que implementa INotifyPropertyChanged.

As propriedades passam a notificar a interface sobre alterações.

Substituição de Event Handlers por Commands:

Os eventos do Code-Behind (Button.Click, etc.) foram substituídos por ICommand (RelayCommand).

Os comandos agora estão nas ViewModels, permitindo testabilidade.

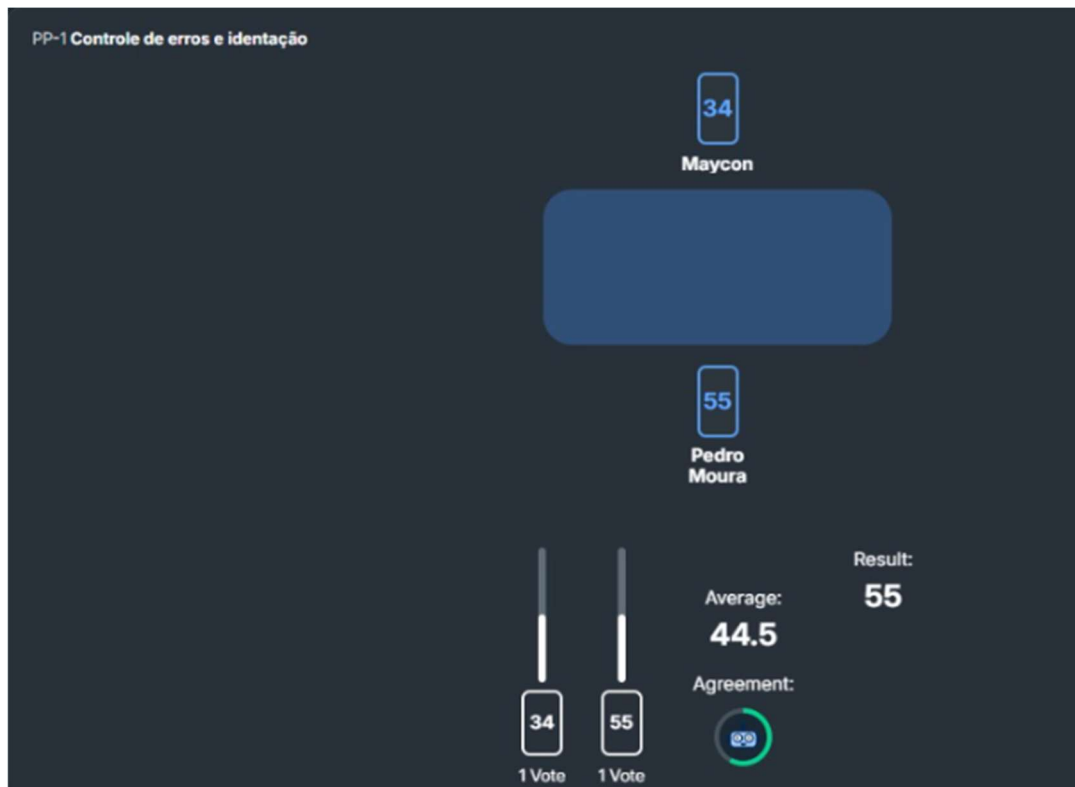
Injeção de Dependência:

Serviços (VisionService, CalibrationService, ClassificationService, ResultadosService) passaram a ser injetados via construtor.

Justificativa: A conversão para MVVM demanda uma abstração maior e compreensão profunda da lógica existente, por isso o tempo estimado é maior. Este padrão garante separação de responsabilidades, testabilidade e manutenibilidade.

CONTROLE DE ERROS E INDENTAÇÃO:

Figura 4 – Votação PlaningPoker



Responsável: Pedro Henrique Moura da Silva

Tempo Estimado: 44 minutos

Descrição: Esta atividade envolveu a implementação de um sistema robusto de tratamento de erros e a padronização da formatação do código:

Tratamento de Exceções:

Implementação de blocos try-catch em operações críticas (comunicação serial, acesso ao banco de dados, processamento de imagens).

Criação do método MensagemAoUsuario() para exibição de erros de forma amigável via MetroDialog.

Criação do serviço ILogService para registro estruturado de logs no banco de dados.

Validações de Estado:

Método ControledeConexao() para validar condições de conexão antes de operações.

Verificações de nullabilidade e estados inválidos.

Padronização de Código:

Aplicação de regras de indentação (4 espaços) em todo o código.

Adição de comentários XML Summary em todas as classes, propriedades e métodos públicos.

Organização do código em regiões (#region) para melhor navegação.

Justificativa: O controle de erros envolve testes e codificação extensiva para validar todas as possibilidades de falha. Um bom tratamento de erros é essencial para garantir a robustez e confiabilidade do sistema.

AJUSTES NO FRONTEND PARA O PADRÃO MVVM:

Figura 5 – Votação PlaningPoker



Responsável: Pedro Henrique Moura da Silva

Tempo Estimado: 75 minutos

Descrição: Esta atividade consistiu na adaptação da interface gráfica para funcionar com o padrão MVVM:

Configuração de Bindings:

Substituição de acesso direto a controles por bindings com as ViewModels.

Configuração de DataContext nas Views.

Implementação de bindings bidirecionais (TwoWay) para propriedades editáveis.

Conversão de Eventos para Commands:

Substituição de eventos Click, Loaded, etc., por bindings de Command e CommandParameter.

Atualização de Controles Visuais:

Ajuste do LiveCharts para bindings com ChartValues.

Ajuste do Gauge (mostrador) para bindings de ângulo, opacidade e texto.

Ajuste do card de resultados para bindings de título, descrição, ícone e cor.

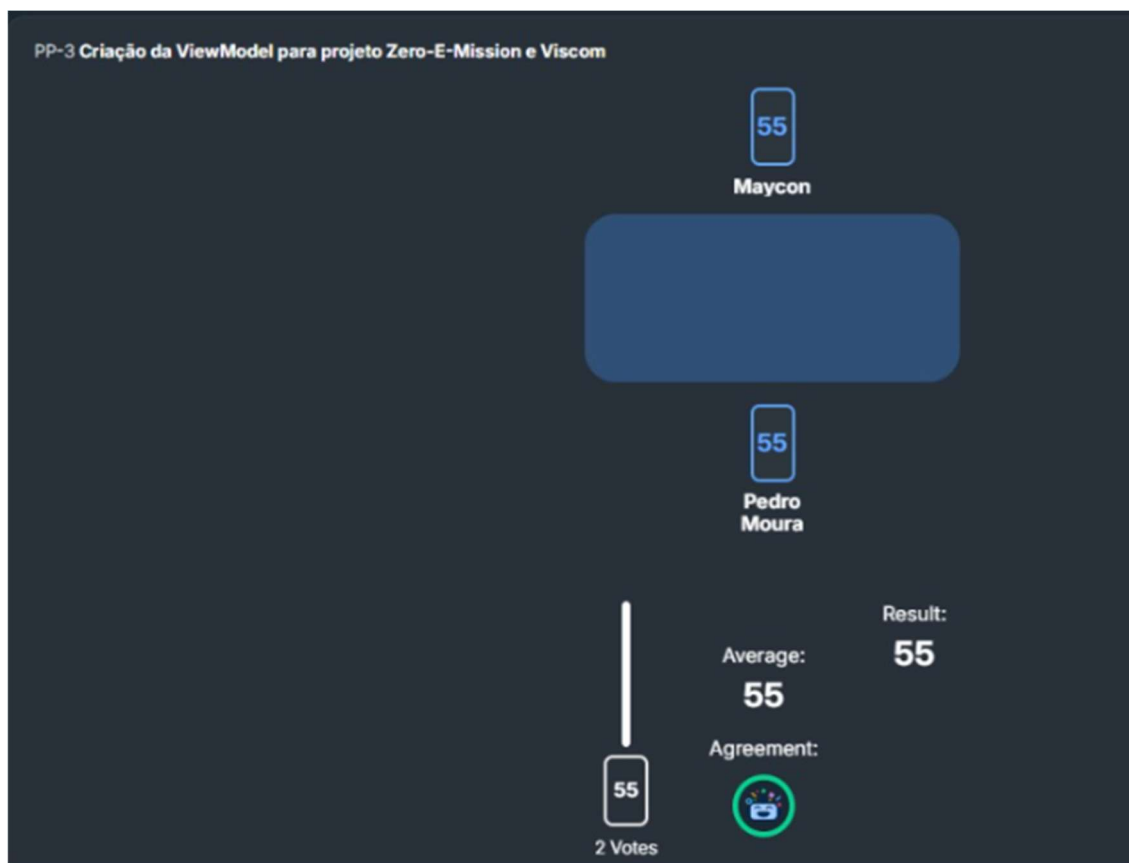
Integração com Serviços de Janela:

Implementação do IWindowService para exibição de janelas modais (gauge) sem quebrar o MVVM.

Justificativa: O ajuste no frontend depende do alinhamento do binding, da verificação de cada botão de ação e da transferência do mecanismo de ação comum para o método MVVM, por isso demanda tempo significativo.

CRIAÇÃO DAS VIEWMODELS:

Figura 6 – Votação PlaningPoker



Responsável: Pedro Henrique Moura da Silva e Maycon Siqueira de Moraes

Tempo Estimado: 55 minutos

Descrição: Esta atividade envolveu a criação das principais ViewModels do sistema:

BaseViewModel:

Classe abstrata base que implementa INotifyPropertyChanged.

Método OnPropertyChanged() com suporte a CallerMemberName.

ZeroEMissionViewModel:

Lógica de comunicação serial com Arduino.

Controle de conexão e desconexão.

Testes dos tanques 1, 2 e 3.

Classificação do combustível por percentual de água.

Atualização do gauge, gráficos e card de resultados.

Persistência de resultados no banco de dados.

Controle de modo simulado.

Acionamento demonstrativo do motor e feedback sonoro.

Integração com ILogService para registro de logs.

ViscomViewModel:

Controle da câmera (iniciar/parar/capturar).

Extração de métricas visuais via VisionService.

Classificação da qualidade via ClassificationService.

Gerenciamento do histórico de análises.

Exportação de dados para CSV.

Persistência de resultados no banco de dados.

Integração com ILogService para registro de logs.

LogsViewModel (opcional):

Visualização de logs armazenados no banco.

Filtros por nível e categoria.

Limpeza de logs antigos.

Justificativa: As ViewModels são o coração do padrão MVVM, responsáveis por toda a lógica de apresentação. A criação destas classes envolve a implementação de comandos, notificações de propriedade, integração com serviços e tratamento de eventos, sendo uma das etapas mais críticas do projeto.

RECURSOS E FERRAMENTAS:

TABELA 7 - CRONOGRAMA

Ferramenta	Finalidade
Visual Studio 2022	IDE para desenvolvimento em C# e WPF
Git / GitHub	Versionamento de código e colaboração
Mermaid	Criação de diagramas UML para documentação

SQLite Browser	Visualização e gerenciamento de banco de dado
Arduino IDE	Programação do firmware
Trello / Notion	Gerenciamento de tarefas e acompanhamento

Fonte: Documentação técnica TCC vector 2026

CONSIDERAÇÕES FINAIS:

O planejamento das atividades de desenvolvimento do sistema Vector – Zero E Mission demonstra que a equipe possui uma visão clara das etapas necessárias para a conclusão do projeto. A estimativa total de aproximadamente 6 horas para as atividades principais reflete o esforço concentrado nas áreas mais críticas:

Conversão para MVVM (90 min): Maior complexidade devido à abstração necessária.

Criação das ViewModels (120 min): Etapa central do desenvolvimento.

Ajustes no frontend (75 min): Dependência de alinhamento de bindings.

O cronograma está alinhado com as melhores práticas de desenvolvimento de software, priorizando a qualidade do código, a robustez e a manutenibilidade do sistema, em conformidade com os objetivos do trabalho de conclusão de curso.

Responsáveis: Pedro Henrique Moura da Silva e Maycon Siqueira de Moraes

Orientador: Frederico Martins Aguiar

Curso: Técnico em Desenvolvimento de Sistemas